

MASTERING NUMPY

INDEXING & SLICING

TECHNICAL FUNDAMENTALS // 2026

THE POWER OF EFFICIENT DATA ACCESS

01

SPECIFIC ACCESS

Instantly retrieve a single element from massive arrays without costly iteration or loops.

02

FEATURE ISOLATION

Extract entire columns or dimensions for specific analysis, a key requirement in data science.

03

SUBSET SLICING

Efficiently grab ranges of data for prototyping, testing, or focused data exploration.

04

ML PREPROCESSING

Isolate features and labels from raw data before training Machine Learning models.

UNDERSTANDING ARRAY INDEXING

THE DEFINITION

Indexing is the process of accessing a **single element** from an array using its exact position.

```
# RULE 01: 0-BASED
```

```
Indexes start from 0.
```

```
# RULE 02: NEGATIVE
```

```
-1 starts from the end.
```

CODE IMPLEMENTATION

```
import numpy as np
```

```
arr = np.array([10, 20, 30, 40, 50])
```

```
# Access first element
```

```
arr[0] -> 10
```

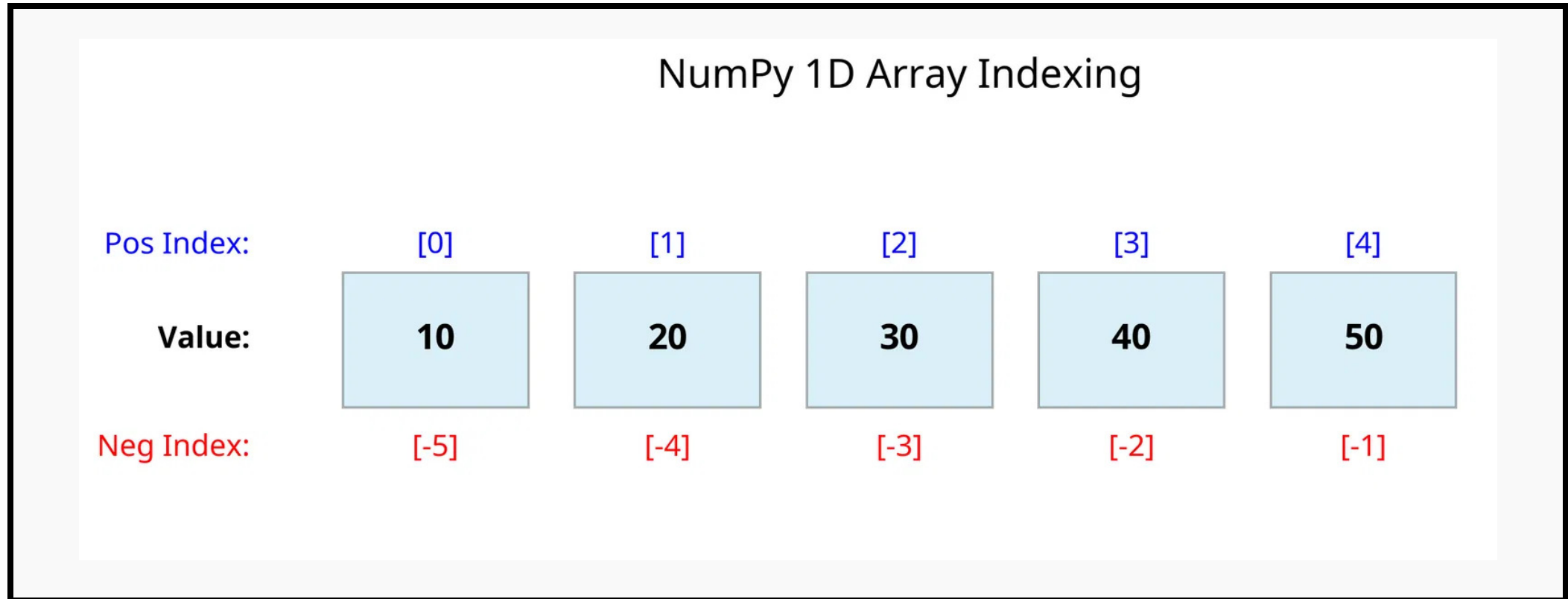
```
# Access third element
```

```
arr[2] -> 30
```

```
# Access last element
```

```
arr[-1] -> 50
```

VISUALIZING 1D ARRAY INDEXING



POSITIVE VS. NEGATIVE INDEXING

INDEXING IN 2D ARRAYS

THE SYNTAX

```
array[row, column]
```

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])
```

```
# Access element 50
```

```
arr[1, 1] -> 50
```

```
# Access element 90
```

```
arr[2, 2] -> 90
```

NumPy 2D Array Indexing [row, col]			
	Col 0	Col 1	Col 2
Row 0	10	20	30
Row 1	40	50	60
Row 2	70	80	90

MATRIX COORDINATES

MASTERING ARRAY SLICING

array[start:stop:step]

PARAMETERS

START

Included index

STOP

Excluded index

STEP

Interval size

1D SLICING EXAMPLES

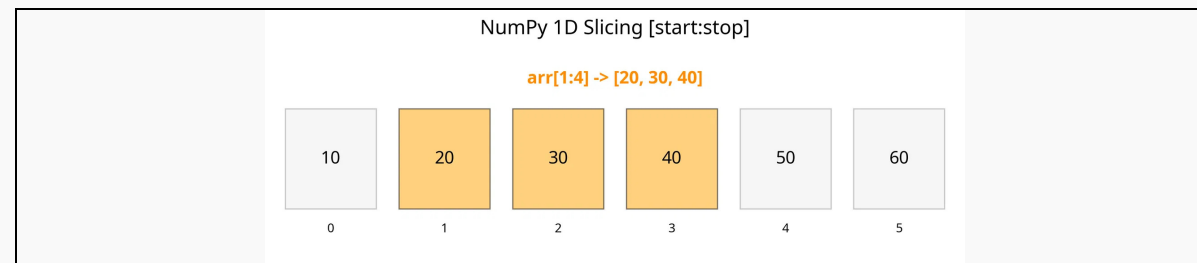
```
arr = np.array([10, 20, 30, 40, 50, 60])
```

```
# Extract range
```

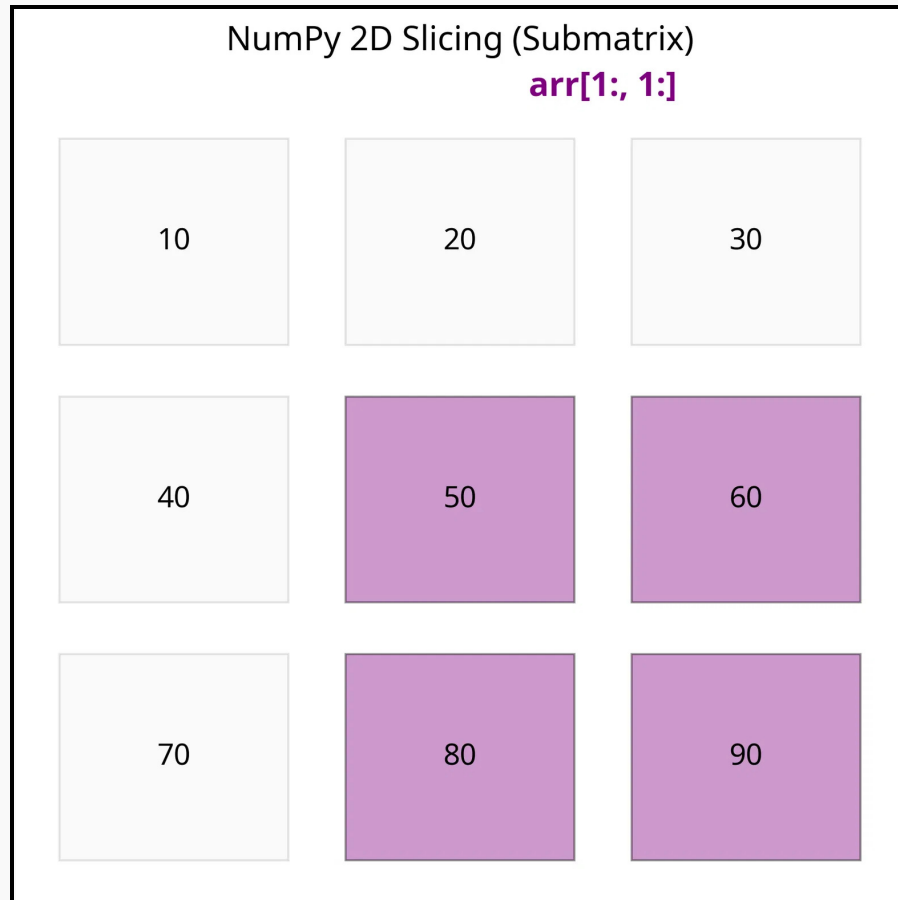
```
arr[1:4]    -> [20, 30, 40]
```

```
# Reverse the entire array
```

```
arr[::-1]   -> [60, 50, 40, 30, 20, 10]
```



ADVANCED SLICING: SUBMATRICES



VISUALIZING `ARR[1:, 1:]`

THE SYNTAX

```
arr[row_slice, col_slice]
```

```
# Extract First Column
```

```
arr[:, 0] -> [10, 40, 70]
```

```
# Extract Submatrix
```

```
arr[1:, 1:]
```

```
-> [[50, 60],  
     [80, 90]]
```

```
# First Two Rows
```

```
arr[:2, :]
```

PRACTICAL CASE: STUDENT DATASETS

THE "STUDENTS" ARRAY STRUCTURE

ID [COL 0]	MARKS [COL 1]	ATTENDANCE [COL 2]
101	85	90
102	78	88
103	92	95
104	65	70

FEATURE EXTRACTION

```
# Get All Marks (Column 1)
students[:, 1] -> [85, 78, 92, 65]

# Get Student IDs (Column 0)
students[:, 0] -> [101, 102, 103, 104]
```

SAMPLE SELECTION

```
# Get First Two Students
students[:2] -> [[101, 85, 90], [102, 78, 88]]

# Marks & Attendance Only
students[:, 1:] -> [[85, 90], [78, 88], ...]
```


KEY TAKEAWAYS & BEST PRACTICES

CORE CONCEPTS

INDEXING VS. SLICING

Indexing retrieves a single element. **Slicing** extracts a subset or range of elements as a new array view.

FLEXIBILITY

NumPy supports positive, negative, and multi-dimensional indexing, providing powerful ways to navigate complex data structures.

PRO TIP:

Slicing returns a **view** of the original array, not a copy. Modifying a slice will change the original data!

MACHINE LEARNING IMPACT

FEATURE SELECTION

Essential for isolating independent variables (X) and dependent variables (y) from raw datasets.

DATA SPLITTING

Used to divide data into training, validation, and test sets with precise control over sample ranges.

PREPROCESSING

Crucial for handling outliers, missing values, and performing batch operations on specific data segments.

THANK YOU

MASTERING NUMPY SERIES

EXPLORE MORE

Advanced Array Operations

RESOURCES

numpy.org/doc

PRESENTED BY

Manus AI